

Práctica 4

Parte PMA

1. Suponga que N clientes llegan a la cola de un banco y que serán atendidos por sus empleados. Analice el problema y defina qué procesos, recursos y canales/comunicaciones serán necesarios/convenientes para resolverlo. Luego, resuelva considerando las siguientes situaciones:

a. Existe un único empleado, el cual atiende por orden de llegada:

```
// ESTRUCTURA
// Hay dos procesos: Cliente, y Empleado.
// Los clientes se ponen en la fila, y esperan a ser atendidos.
// Mientras que el empleado los va atendiendo uno a uno, por orden de llegada.
a.
// Se va a tener que utilizar por lo menos un canal para que los clientes
// puedan encolar sus ids (¿hace falta otro canal para que los clientes sepan
// cuando irse? → CONSULTA)

// Voy a utilizar el canal 'canal_ids_clientes' para pasar los ids del cliente
// al empleado, y no me tengo que preocupar por el orden ya que este se encuentra
// implícito en el canal. [Filmina 8 y 9 de la Teoría 5]

// Un canal se declara con su nombre y el tipo de dato que acepta
chan CANAL_ID_CLIENTES(int);
chan CANAL_RECIBO[N](recibo);

// Para enviar una variable (un mensaje), se utiliza "send <nombre_canal>(me
```

```

nsaje)"
process Cliente [Id: 1..N] {
    recibo aux_recibo;
    send CANAL_ID_CLIENTES(Id);
    receive CANAL_RECIBO[Id](aux_recibo);
}

process Empleado {
    int id_auxiliar;
    // Asumamos que van llegando cliente para siempre
    while (True) {
        // Para recibir una variable, se utiliza
        // "receive <nombre_canal>(variable_receptora)"
        receive CANAL_ID_CLIENTES(id_auxiliar);
        recibo aux_recibo = Atender(id_auxiliar);
        send CANAL_RECIBO[id_auxiliar](aux_recibo);
    }
}

```

[Corregido]

b. Ídem a) pero considerando que hay 2 empleados para atender, ¿qué debe modificarse en la solución anterior?

```

// Ahora se me ocurre que podrían hacerse dos cosas:
// 1. Crear un canal para cada empleado, pero esto tiene como punto en contr
a que
// cada proceso Cliente tendría que saber de antemano el número del emple
ado
// con el que tiene que comunicarse, y además de esto, sería posible que ha
ya
// demora innecesaria, ya que un proceso podría encolarse en la cola de un
// empleado que actualmente está ocupado, mientras que el otro empleado
no.
// 2. Que los dos empleados utilicen el mismo canal. Podría surgir el problema
// que aparece cuando hay mas de un receptor por canal, pero en este caso

```

```

no
//  creo que haya problema, ya que las operaciones son atómicas, y cada em
pleado
//  irá desencolando un id cuando haya y cuando pueda.
// Por lo tanto, no se modifica nada con respecto a la solución anterior.

chan CANAL_ID_CLINTES(int);
chan CANAL_RECIBO[N](recibo);

process Cliente [Id: 1..N] {
    send CANAL_ID_CLINTES(Id);
    recibo aux_recibo;
    receive CANAL_RECIBO[Id](aux_recibo);
}

process Empleado [Id: 1..2] {
    int id_auxiliar;
    while (True) {
        receive CANAL_ID_CLINTES(id_auxiliar);
        recibo aux_recibo = Atender(id_auxiliar);
        send CANAL_RECIBO[id_auxiliar](aux_recibo);
    }
}

```

[Corregido]

c. Ídem b) pero considerando que, si no hay clientes para atender, los empleados realizan tareas administrativas durante 15 minutos. ¿Se puede resolver sin usar procesos adicionales? ¿Qué consecuencias implicaría?

```

// Ahora si surge el problema que ocurre cuando hay mas de un receptor en el
mismo canal.
// Puede ocurrir que los dos empleados evalúen la condición de si el canal est
á vacío
// en el mismo instante, y entonces ambos entren al bloque, uno de ellos obte

```

```
nga el
// id, y el otro se haya puesto en el receive, ahora en la espera por el próximo i
d
// que llegue, cuando en realidad lo que debería haber hecho era dormir por 1
5 minutos.
```

```
// Es imposible
```

```
chan CANAL_ID_CLIENTES(int);
chan CANAL_COORDINADOR(int);
chan CANAL_EMPLEADO[2](int);
chan CANAL_RECIBO[N](recibo);
```

```
process Cliente [Id: 1..N] {
    recibo aux_recibo;
    send CANAL_ID_CLIENTES(Id);
    receive CANAL_RECIBO[Id](aux_recibo);
}
```

```
process Coordinador {
    int id_cli;
    int id_empl;
    while (True) {
        receive CANAL_COORDINADOR(id_empl);
        if (not empty(CANAL_ID_CLIENTES))
            receive CANAL_ID_CLIENTES(id_cli);
        else
            int id_cli = -1;
        send CANAL_EMPLEADO[id_empl](id_cli);
    }
}
```

```
process Empleado [Id: 1..2] {
    int id_auxiliar;
    while (True) {
```

```

        send CANAL_COORDINADOR(Id);
        int id_cli;
        receive CANAL_EMPLEADO[Id](id_cli);
        if (id_cli <> -1) aux_recibo = Atender(id_cli); send CANAL_RECIBO[id
        _auxiliar](aux_recibo);
        else delay(900); // 15 * 60
    }
}

```

[Corregido]

2. Se desea modelar el funcionamiento de un banco en el cual existen 5 cajas para realizar pagos. Existen P clientes que desean hacer un pago. Para esto, cada uno selecciona la caja donde hay menos personas esperando; una vez seleccionada, espera a ser atendido. En cada caja, los clientes son atendidos por orden de llegada por los cajeros. Luego del pago, se les entrega un comprobante.

```

process Cliente [Id: 1..P] {
    send CANAL_FILA(Id);
    receive CANAL_COMPROBANTE[Id](comprob);
}

process AdminCajas {
    int id_cli;
    int id_imp;
    int[] cant_esperando_cajas = ([5], 0);
    while (True) {
        if (not empty(CANAL_FILA))
            receive CANAL_FILA(id_cli);

```

```

        int id_imp = ObtenerMinimo(cant_esperando_cajas);
        cant_esperando_cajas[id_imp]++;
        send CANAL_EMPLEADO[id_imp](id_cli);

    else (not empty(CANAL_TERMINARON))
        receive CANAL_TERMINARON(id_imp);
        cant_esperando_cajas[id_imp]--;
    }
}

process Caja [Id: 1..5] {
    int id_cli;

    while (True) {
        receive CANAL_EMPLEADO[Id](id_cli);
        comprobante comprob = Atender(id_cli);
        send CANAL_COMPROBANTE[id_cli](comprob);
        send CANAL_TERMINARON(Id);
    }
}

```

[Corregido]

3. Se debe modelar el funcionamiento de una casa de comida rápida, en la cual trabajan 2 cocineros y 3 vendedores, y que debe atender a C clientes.

```

chan CANAL_PEDIDO(int, pedido);
chan CANAL_ENTREGA[C](entrega);
chan CANAL_VENDEDOR_COORD(int);
chan CANAL_COORD_VENDEDOR[3](int, pedido);
chan CANAL_VENDEDOR_COCINEROS(int, pedido);

process Cliente [Id: 1..C] {

```

```

    pedido aux_pedido;
    entrega aux_entrega;
    send CANAL_PEDIDO(Id, pedido);
    receive CANAL_ENTREGA[Id](aux_entrega);
}

// Agrego este proceso ya que los dos utiliza un solo canal, y si no hay pedido
// s en
// el canal, el proceso que esté libre en ese momento se tiene que ir a dormir
process CoordinadorVendedor {
    int id_cli;
    int id_vend;
    pedido pedido_cli;
    while (True) {
        // Esperar a que llegue un vendedor
        receive CANAL_VENDEDOR_COORD(id_vend);
        // Revisar si llegó algún cliente
        if (not empty(CANAL_PEDIDO))
            receive CANAL_PEDIDO(id_cli, pedido_cli);
        else
            id_cli = -1;
            pedido_cli = null;
        // Hacer que el cliente pase con dicho vendedor
        send CANAL_COORD_VENDEDOR[id_vend](id_cli, pedido_cli);
    }
}

process Vendedor [Id: 1..3] {
    int id_cli;
    pedido pedido_cli;
    while (True) {
        // Avisar que está disponible para recibir algún cliente
        send CANAL_VENDEDOR_COORD(Id);
        // Esperar respuesta del coordinador
        receive CANAL_COORD_VENDEDOR[Id](id_cli);
        if (id_cli <> -1)

```

```

        send CANAL_VENDEDOR_COCINEROS(id_cli, pedido_cli);
    else
        int tiempo = Random(60, 180);
        delay(tiempo);
    }
}

process Cocinero [Id: 1..2] {
    int id_cli;
    pedido pedido_cli;
    while (True) {
        // Quedarse esperando a que llegue algún pedido desde los vended
        ores
        receive CANAL_VENDEDOR_COCINEROS(id_cli, pedido_cli);
        // Realizar pedido
        entrega entrega_cli = Cocinar(pedido_cli);
        // Entregárselo al cliente
        send CANAL_ENTREGA[id_cli](entrega_cli);
    }
}

```

[Corregido]

4. Simular la atención en un locutorio con 10 cabinas telefónicas, el cual tiene un empleado que se encarga de atender a N clientes. Al llegar, cada cliente espera hasta que el empleado le indique a qué cabina ir, la usa y luego se dirige al empleado para pagarle. El empleado atiende a los clientes en el orden en que hacen los pedidos. A cada cliente se le entrega un ticket factura por la operación.

a) Implemente una solución para el problema descrito.

```

chan CANAL_COLA(int);
chan CANAL_CLI_EMPL[N](int);
chan CANAL_EMPL_CLI_FACTURA(ticket);

```



```

process Cliente [Id: 1..N] {
    int num_cabina;
    ticket factura;
    // Encolarse en la fila (el buffer)
    send CANAL_COLA(Id);
    // El empleado le dice a qué cabina debe dirigirse
    // Utilizar cabina
    send CANAL_CLI_TERMINO(Id);
    receive CANAL_EMPL_CLI_FACTURA[Id](factura);
}

process Empleado {
    cola<int> cola_cabinas = {[10]};
    cola<Ticket> tickets_entregados;
    int clientes_usando = 0;
    int id_cli;
    ticket factura;
    while (True) {
        if (not empty(CANAL_COLA) && clientes_usando < 10)
            receive CANAL_COLA(id_cli);
            clientes_usando++;
            int num_cabina = pop(cola_cabinas);
            send CANAL_CLI_EMPL[id_cli](num_cabina);
        else (not empty(CANAL_EMPL_CLI_FACTURA) && clientes_usando >
0)
            receive CANAL_CLI_TERMINO(id_cli);
            ticket factura = GenerarFactura();
            send CANAL_EMPL_CLI_FACTURA[id_cli](factura);
            push(tickets_entregados, factura);
            clientes_usando--;
            push(cola_cabinas, num_cabina);
        }
    }
}

```

[Corregido]

b) Modifique la solución implementada para que el empleado dé prioridad a los que terminaron de usar la cabina sobre los que están esperando para usarla.

```
chan CANAL_COLA(int);
chan CANAL_CLI_EMPL[N](int);
chan CANAL_EMPL_CLI_FACTURA(ticket);

process Cliente [Id: 1..N] {
    int num_cabina;
    ticket factura;
    // Encolarse en la fila (el buffer)
    send CANAL_COLA(Id);
    // El empleado le dice a qué cabina debe dirigirse
    // Utilizar cabina
    send CANAL_CLI_TERMINO(Id);
    receive CANAL_EMPL_CLI_FACTURA[Id](factura);
}

process Empleado {
    cola<int> cola_cabinas = {[10]};
    cola<Ticket> tickets_entregados;
    int clientes_usando = 0;
    int id_cli;
    ticket factura;
    while (True) {
        if (not empty(CANAL_EMP_CLI_TERMINO) && clientes_usando > 0)
            receive CANAL_CLI_TERMINO(id_cli);
            ticket factura = GenerarFactura();
            send CANAL_EMPL_CLI_FACTURA[id_cli](factura);
            push(tickets_entregados, factura);
            clientes_usando--;
            push(cola_cabinas, num_cabina);
        else (not empty(CANAL_COLA) && empty(CANAL_EMP_CLI_TERMINO) && clientes_usando < 10)
            receive CANAL_COLA(id_cli);
```

```

        clientes_usando++;
        int num_cabina = pop(cola_cabinas);
        send CANAL_CLI_EMPL[id_cli](num_cabina);
    }
}

```

[Corregido]

5. Resolver la administración de 3 impresoras de una oficina. Las impresoras son usadas por N administrativos, los cuales están continuamente trabajando y cada tanto envían documentos a imprimir. Cada impresora, cuando está libre, toma un documento y lo imprime, de acuerdo con el orden de llegada

a) Implemente una solución para el problema descrito

```

chan CANAL_RECIBIR_DOCUMENTO(documento, int);
chan IMPRESION[N](impresion);

process Administrativo [Id: 1..N] {
    documento doc;
    impresion impr;
    // Los administrativos están continuamente trabajando
    while (True) {
        doc = HacerTarea();
        send CANAL_DOCUMENTO(doc, Id);
        // ¿Tienen que esperar a recibir la impresión?
        if (not empty(CANAL_IMPRESION[Id]))
            receive CANAL_IMPRESION[Id](impr);
    }
}

```

```

process Impresora [Id: 1..3] {
    int id_cli;
    documento doc;
    impresion impr;
    while (True) {
        receive CANAL_RECIBIR_DOCUMENTO(id_cli, doc);
        impr = Imprimir(doc);
        send CANAL_IMPRESION[id_cli](doc);
    }
}

```

[Corregido]

b) Modifique la solución implementada para que considere la presencia de un director de oficina que también usa las impresas, el cual tiene prioridad sobre los administrativos.

```

chan CANAL_DOCUMENTO(documento, int);
chan CANAL_AVISAR(int);
chan CANAL_RECIBIR_DOCUMENTO[3](documento, int);
chan IMPRESION[N](impresion);

process Administrativo [Id: 1..N] {
    documento doc;
    impresion impr;
    // Los administrativos están continuamente trabajando
    while (True) {
        doc = HacerTarea();
        send CANAL_DOCUMENTO(doc, Id);
        sen CANAL_AVISAR(1);
    }
}

process Director {
    documento doc;

```

```

    impresion impr;

    while (True) {
        doc = HacerTarea();
        send CANAL_DIRECTOR(doc, Id);
        send CANAL_AVISAR(1);
    }
}

process Coordinador {
    int idI, idAux;
    while (true) {
        receive CANAL_AVISAR();
        receive CANAL_IMPRESORA(idI);

        if (not CANAL_DIRECTOR.isEmpty()) → {
            receive CANAL_DIRECTOR(documento, idD);
        }
        □ (not CANAL_DOCUMENTO.isEmpty() && CANAL_DIRECTOR.isEmpty())
→ {
        receive CANAL_DOCUMENTO(documento, idAux);
    }

    send CANAL_RECIBIR_DOCUMENTO[idI](documento, idAux);
}
}

process Impresora [Id: 1..3] {
    int id_cli;
    documento doc;
    impresion impr;
    while (True) {
        send CANAL_AVISO_IMPRESORA(Id);
        receive CANAL_RECIBIR_DOCUMENTO[Id](id_cli, doc);
        impr = Imprimir(doc);
    }
}

```

```
}  
}
```

[Corregido]

c) Modifique la solución (a) considerando que cada administrativo imprime 10 trabajos y que todos los procesos deben terminar su ejecución.

```
chan CANAL_DOCUMENTO(documento, int);  
chan CANAL_RECIBIR_DOCUMENTO(documento, id);  
chan CANAL_AVISO_DISPONIBLE(int);  
chan IMPRESION[N](impresion);  
  
process Administrativo [Id: 1..N] {  
    documento doc;  
    impresion impr;  
    // Los administrativos están continuamente trabajando  
    for (int i = 0; i < 10; i++) {  
        doc = HacerTarea();  
        send CANAL_DOCUMENTO(doc, Id);  
        // ¿Tienen que esperar a recibir la impresión?  
        if (not empty(CANAL_IMPRESION[Id])  
            receive CANAL_IMPRESION[Id](impr);  
    }  
}  
  
// Este proceso va tanto a administrar el ingreso de los documentos  
// como contarlos  
process BufferImpresora {  
    int id_cli;  
    documento doc;  
    int contador_documentos_a_imprimir = 10 * N;  
  
    while (True) {  
        receive CANAL_AVISO_DISPONIBLE();
```

```

        if (contador_documentos_a_imprimir > 0)
            receive CANAL_DOCUMENTO(id_cli, doc);
            contador_documentos_a_imprimir--;
        else
            id_cli = -1;
            doc = null;
            send CANAL_RECIBIR_DOCUMENTO(id_cli, doc);
    }
}

process Impresora [Id: 1..3] {
    int id_cli;
    documento doc;
    impresion impr;
    bool termino = false;
    while (not termino) {
        send CANAL_AVISO_DISPONIBLE(1);
        receive CANAL_DOCUMENTO(id_cli, doc);
        if (id_cli <> -1)
            impr = Imprimir(doc);
            send CANAL_IMPRESION[id_cli](doc);
        else
            termino = true;
    }
}

```

[Corregido]

d) Modifique la solución (b) considerando que tanto el director como cada administrativo imprimen 10 trabajos y que todos los procesos deben terminar su ejecución

```

chan CANAL_DOCUMENTO(documento, int);
chan CANAL_AVISAR(int);
chan CANAL_RECIBIR_DOCUMENTO[3](documento, int);

```

```

chan IMPRESION[N](impresion);

process Administrativo [Id: 1..N] {
    documento doc;
    impresion impr;
    // Los administrativos están continuamente trabajando
    for (int i = 0; i < 10; i++) {
        doc = HacerTarea();
        send CANAL_DOCUMENTO(doc, Id);
        send CANAL_AVISAR(1);
    }
}

process Director {
    documento doc;
    impresion impr;

    for (int i = 0; i < 10; i++) {
        doc = HacerTarea();
        send CANAL_DIRECTOR(doc, Id);
        send CANAL_AVISAR(1);
    }
}

process Coordinador {
    int idI, idAux;
    // Todos los procesos hacen 10 trabajos
    int contador_documentos_a_imprimir = 10 * (N + 1);
    while (true) {
        if (contador_documentos_a_imprimir > 0)
            receive CANAL_AVISAR();
            receive CANAL_IMPRESORA(idI);

        if (not CANAL_DIRECTOR.isEmpty()) → {
            receive CANAL_DIRECTOR(documento, idD); }
        □ (not CANAL_DOCUMENTO.isEmpty() && CANAL_DIRECTOR.isEmp

```



```

ty() → {
    receive CANAL_DOCUMENTO(documento, idAux);
}
else
    receive CANAL_IMPRESORA(idI);
    idAux = -1;
    documento = null;

    send CANAL_RECIBIR_DOCUMENTO[idI](documento, idAux);
}
}

process Impresora [Id: 1..3] {
    int id_cli;
    documento doc;
    impresion impr;
    bool termino = false;
    while (not termino) {
        send CANAL_AVISO_IMPRESORA(Id);
        receive CANAL_RECIBIR_DOCUMENTO[Id](doc, id_cli);
        if (id_cli <> -1)
            impr = Imprimir(doc);
        else
            termino = true;
    }
}

```

[Corregido]

e) Si la solución al ítem d) implica realizar Busy Waiting, modifícala para evitarlo.

- Primero tendría que corregir los dos últimos incisos

Parte PMS

1. Suponga que existe un antivirus distribuido que se compone de R procesos robots Examinadores y 1 proceso Analizador. Los procesos Examinadores están buscando continuamente posibles sitios web infectados; cada vez que encuentran uno avisan la dirección y luego continúan buscando. El proceso Analizador se encarga de hacer todas las pruebas necesarias con cada uno de los sitios encontrados por los robots para determinar si están o no infectados

a. Analice el problema y defina qué procesos, recursos y comunicaciones serán necesarios/convenientes para resolverlo.

- Vamos a necesitar R procesos examinadores, 1 proceso analizador, y una forma de comunicar R procesos con el único analizador disponible (¿un arreglo?)

b. Implemente una solución con PMS sin tener en cuenta el orden de los pedidos

```
// VERSIÓN 1 - No importa el tiempo que espere cada examinador
process Examinador [1..R] {
    sitio sitio_inf;
    while (True) {
        // Buscar sitio web infectado
        sitio_inf = buscarSitiInfectado();
        // Avisarle al analizador
        Analizador!(sitio_inf);
        // ¿Tiene que recibir la respuesta?
    }
}

process Analizador {
    sitio sitio_inf;
    bool resp_inf;
```

```

while (True) {
    // Recibir sitio web infectado de cualquier examinador
    Examinador[*]?(sitio_inf);
    // Determinar si está infectado o no
    resp_inf = analizarSitio(sitio_inf);
    // ¿Tiene que enviarle la respuesta al examinador?
}
}

// VERSIÓN 2 - Se intenta agilizar un poco el tiempo de espera del examinador
process Examinador [1..R] {
    sitio sitio_inf;
    while (True) {
        sitio_inf = buscarSitiInfectado();
        Analizador!(sitio_inf);
    }
}

// "Pasamanos"
process BufferExaminador {
    cola<sitio> cola_sitios;
    sitio sitio_inf;
    while (True) {
        do Examinador[*]?(sitio_inf); push(cola_sitios, sitio_inf);
        □ (not empty(cola_sitios)); Analizador?aviso() →
            Analizador!recibo(pop(cola_sitio));
    }
}

process Analizador {
    sitio sitio_inf;
    bool resp_inf;
    while (True) {
        BufferExaminador!aviso(1);
        BufferExaminador?recibo(sitio_inf);
        resp_inf = analizarSitio(sitio_inf);
    }
}

```

```
}  
}
```

[Corregido] → Versión 2

c. Modifique el inciso (b) para que el Analizador resuelva los pedidos en el orden en que se hicieron.

```
// LA COLA DEL BUFFER DE POR SI IMPLEMENTA ESTO CREO  
process Examinador [1..R] {  
    sitio sitio_inf;  
    while (True) {  
        sitio_inf = buscarSitiInfestado();  
        Analizador!(sitio_inf);  
    }  
}  
  
// "Pasamanos"  
process BufferExaminador {  
    cola<sitio> cola_sitios;  
    sitio sitio_inf;  
    while (True) {  
        do Examinador[*]?(sitio_inf); push(cola_sitios, sitio_inf);  
        □ (not empty(cola_sitios)); Analizador?aviso() →  
            Analizador!recibo(pop(cola_sitio));  
    }  
}  
  
process Analizador {  
    sitio sitio_inf;  
    bool resp_inf;  
    while (True) {  
        BufferExaminador!aviso(1);  
        BufferExaminador?recibo(sitio_inf);  
        resp_inf = analizarSitio(sitio_inf);  
    }  
}
```

```
}  
}
```

[Corregido]

2. En un laboratorio de genética veterinaria hay 3 empleados. El primero de ellos continuamente prepara las muestras de ADN; cada vez que termina, se la envía al segundo empleado y vuelve a su trabajo. El segundo empleado toma cada muestra de ADN preparada, arma el set de análisis que se deben realizar con ella y espera el resultado para archivarlo. Por último, el tercer empleado se encarga de realizar el análisis y devolverle el resultado al segundo empleado

```
// Como el enunciado dice que "el primero de ellos continuamente  
// prepara las muestras de ADN", deduzco que no tiene que hacer  
// Busy Waiting, o lo menos posible  
process Preparador {  
    muestraADN muestra;  
    while (True) {  
        muestra = prepararMuestraADN();  
        BufferPreparador!(muestra);  
    }  
}  
  
process BufferPreparador {  
    muestraADN muestra;  
    cola<muestraADN> cola_muestras;  
    while (True) {  
        do Preparador?(muestra); → push(cola_muestras, muestra);  
        □ (not empty(cola_muestras)); Armador?aviso() →  
            Armador!recibo(pop(cola_muestras));  
    }  
}
```

```

}

process Armador {
    muestraADN muestra;
    setAnalisis set;
    resultado res;
    while (True) {
        BufferPreparador!aviso();
        BufferPreparador?recibo(muestra);
        set = armarSetAnalisis(muestra);
        Realizador!envioSet(set);
        Realizar?envioResultado(res);
        // Simular que se archiva el resultado
        archivar(res);
    }
}

process Realizador {
    setAnalisis set;
    resultado res;
    while (True) {
        Armador?envioSet(set);
        res = realizarAnalisis(set);
        Armador!envioResultado(res);
    }
}

```

[Corregido]

3. En un examen final hay N alumnos y P profesores. Cada alumno resuelve su examen, lo entrega y espera a que alguno de los profesores lo corrija y le indique la nota. Los profesores corrigen los exámenes

respetando el orden en que los alumnos van entregando.

a. Considerando que $P=1$.

```
process Alumno [Id: 1..N] {
    examen exam;
    int nota;
    exam = resolverExamen();
    BufferExamenes!entregar_examen(exam, Id);
    Profesor?entregar_nota(nota);
}

// Los exámenes se corrigen respetando el orden en que se entregan
process BufferExamenes {
    int id_alum;
    int exámenes_por_entregar = N;
    examen exam_alum;
    cola<examen, int> cola_exámenes;

    while (exámenes_por_entregar > 0) {
        do Alumno[*]?entregar_examen(exam_alum, id_alum); →
            push(cola_exámenes, exam_alum, id_alum);
        □ (not empty(cola_exámenes)); Profesor?aviso() →
            Profesor!envio_examen(pop(cola_exámenes));
            exámenes_por_entregar--;
    }
}

// "Los profesores corrigen los exámenes"
process Profesor {
    int id_alum;
    int nota;
    examen exam_alum;
```

```

    for (int i = 0; i < N; i++) {
        BufferExamenes!aviso(1);
        BufferExamenes?envio_examen(exam_alum, id_alum);
        nota = corregirExamen(exam_alum);
        Alumno[Id]!entregar_nota(nota);
    }
}

```

[Corregido]

b. Considerando que $P > 1$.

```

process Alumno [Id: 1..N] {
    examen exam;
    int nota;
    // "Cada alumno resuelve su examen"
    exam = resolverExamen();
    // "lo entrega"
    BufferExamenes!entregar_examen(exam, Id);
    // "y espera a que alguno de los profesores lo corrija y le indique la nota"
    Profesor?entregar_nota(nota);
}

// Los exámenes se corrijan respetando el orden en que se entregan
process BufferExamenes {
    int id_alum;
    int id_prof;
    int examenes_por_entregar = N;
    examen exam_alum;
    cola<examen, int> cola_examenes;

    while (examenes_por_entregar > 0) {
        do Alumno[*]?entregar_examen(exam_alum, id_alum); →
            push(cola_examenes, exam_alum, id_alum);
        □ (not empty(cola_examenes)); Profesor[*]?aviso(id_prof) →
            Profesor[id_prof]!envio_examen(pop(cola_examenes));
    }
}

```



```

        examenes_por_entregar--;
    }

    // Avisarle a los profes que ya no hay mas alumnos ¿Busy Waiting?
    for (int i = 0; i < 3; i++)
        Profesor!envio_examen(null, -1);
}

// "Los profesores corrigen los exámenes"
process Profesor [Id: 1..P] {
    int id_alum;
    int nota;
    examen exam_alum;
    bool termino = false;

    while (not termino) {
        BufferExamenes!aviso(Id);
        BufferExamenes?envio_examen(exam_alum, id_alum);
        if (id_alum <> -1)
            nota = corregirExamen(exam_alum);
            Alumno[Id]!entregar_nota(nota);
        else
            termino = true;
    }
}

```

[Corregido]

c. Ídem b) pero considerando que los alumnos no comienzan a realizar su examen hasta que todos hayan llegado al aula.

```

process Alumno [Id: 1..N] {
    examen exam;
    int nota;
    // " Los alumnos no comienzan a realizar su examen hasta que todos hay
an

```

```

    // Llegado al aula."
    BarreraAlumnos!aviso(1);
    BarreraAlumnos?permitir_paso();
    // "Cada alumno resuelve su examen"
    exam = resolverExamen();
    // "lo entrega"
    BufferExamenes!entregar_examen(exam, Id);
    // "y espera a que alguno de los profesores lo corrija y le indique la nota"
    Profesor?entregar_nota(nota);
}

// Implementar una barrera para alumnos
process BarreraAlumnos {
    // Esperar que lleguen los N alumnos
    for (int i = 0; i < N; i++)
        Alumno[*]?aviso()

    // Avisarle a todos que pueden pasar ya
    for (int i = 0; i < N; i++)
        Alumno[i]!permitir_paso(1);
}

// Los exámenes se corrigen respetando el orden en que se entregan
process BufferExamenes {
    int id_alum;
    int id_prof;
    int examenes_por_entregar = N;
    examen exam_alum;
    cola<examen, int> cola_examenes;

    while (examenes_por_entregar > 0) {
        do Alumno[*]?entregar_examen(exam_alum, id_alum); →
            push(cola_examenes, exam_alum, id_alum);
        □ (not empty(cola_examenes)); Profesor[*]?aviso(id_prof) →
            Profesor[id_prof]!envio_examen(pop(cola_examenes));
            examenes_por_entregar--;
    }
}

```

```

    }

    // Avisarle a los profes que ya no hay mas alumnos ¿Busy Waiting?
    for (int i = 0; i < 3; i++)
        Profesor!envio_examen(null, -1);
}

// "Los profesores corrigen los exámenes"
process Profesor [Id: 1..P] {
    int id_alum;
    int nota;
    examen exam_alum;
    bool termino = false;

    while (not termino) {
        BufferExamenes!aviso(Id);
        BufferExamenes?envio_examen(exam_alum, id_alum);
        if (id_alum <> -1)
            nota = corregirExamen(exam_alum);
            Alumno[Id]!entregar_nota(nota);
        else
            termino = true;
    }
}

```

[Corregido]

4. En una exposición aeronáutica hay un simulador de vuelo (que debe ser usado con exclusión mutua) y un empleado encargado de administrar su uso. Hay P personas que esperan a que el empleado lo deje acceder al simulador, lo usa por un rato y se retira.

a. Implemente una solución donde el empleado sólo se ocupa de garantizar la exclusión mutua (sin importar el orden).

```

process Persona [Id: 1..P] {
    // Le avisa al empleado que quiere pasar
    Empleado!quiere_pasar(Id);
    // Espera a que el empleado lo llame
    Empleado?puede_pasar();
    // Utiliza el simulador de vuelo
    utilizarSimuladorVuelo();
    // Avisarle al empleado que ya terminó
    Empleado!termino(1);
}

process Empleado {
    int id_per;
    // "Hay P personas que esperan a que el empleado lo deje acceder al simulador"
    // No importa el orden
    for (int i = 0; i < P; i++) {
        Persona[*]?quiere_pasar(id_per);
        Persona[id_per]!puede_pasar(1);
        Persona[id_per]?termino();
    }
}

```

[Corregido]

b. Modifique la solución anterior para que el empleado los deje acceder según el orden de su identificador (hasta que la persona i no lo haya usado, la persona $i+1$ debe esperar).

```

process Persona [Id: 1..P] {
    Empleado?puede_pasar();
    utilizarSimuladorVuelo();
    Empleado!termino(1);
}

```

```

process Empleado {
    // Según el orden de su identificador
    for (int i = 0; i < P; i++) {
        Persona[i]!puede_pasar(1);
        Persona[i]?termino();
    }
}

```

[Corregido]

c. Modifique la solución a) para que el empleado considere el orden de llegada para dar acceso al simulador.

```

process Persona [Id: 1..P] {
    BufferPersona!quiere_pasar(Id);
    Empleado?puede_pasar();
    utilizarSimuladorVuelo();
    Empleado!termino(1);
}

// Para que el empleado considere el orden de llegada, es
// necesario meter un proceso Buffer. Una vez que haya
// pushado los P ids de las personas, entonces el proceso
// termina, porque no tiene que hacer nada mas
process BufferPersona {
    cola<int> cola_ids;
    int id_per;
    int cont_personas_faltan = P;

    while (cont_personas_faltan > 0) {
        do Persona[*]?quiere_pasar(id_per) → push(cola_ids, id_per);
        □ (not empty(cola_ids)) Empleado?aviso() →
            Empleado!enviar_id(pop(cola_ids));
            cont_personas_faltan--;
    }
}

```

```

process Empleado {
    int id_per;
    for (int i = 0; i < P; i++) {
        BufferPersona!aviso(1);
        BufferPersona?enviar_id(id_per);
        Persona[id_per]!puede_pasar(1);
        Persona[id_per]?termino();
    }
}

```

[Corregido]

5. En un estadio de fútbol hay una máquina expendedora de gaseosas que debe ser usada por E Espectadores de acuerdo con el orden de llegada. Cuando el espectador accede a la máquina en su turno usa la máquina y luego se retira para dejar al siguiente. Nota: cada Espectador una sólo una vez la máquina.